

decision (Figure 2).

A threshold that is set too low may reject valid matches, reducing the cache hit ratio through false negatives. Conversely, a threshold that is too high increases the likelihood of returning incorrect responses by treating unrelated queries as equivalent.

A practical approach is to benchmark against representative datasets. Tools such as the Retrieval

Optimiser can evaluate different threshold values, helping identify settings that maximise retrieval accuracy while reducing incorrect cache hits.

Another recommended practice is to apply an intent classification filter before accessing the semantic cache. This filter determines whether an incoming request is suitable for caching, preventing personalised or frequently changing queries from being stored unnecessarily.

Redis also provides an open source semantic router, enabling intent classification through vector search without requiring an additional LLM call.

Design principle 3: Cache design and eviction

Building a semantic cache involves more than storing responses. Since Redis is an in-memory database, memory must be managed carefully to ensure the cache remains efficient as applications scale (Figure 3).

One of the first considerations is selecting an appropriate eviction policy. When memory usage becomes high, older or less frequently accessed entries should be removed to make room for new ones. Common approaches include Least Recently Used (LRU) and Least Frequently Used (LFU) eviction policies, both of which help maintain cache performance without requiring excessive memory resources.

Setting an appropriate Time to Live (TTL) is equally important. The TTL should reflect how frequently the underlying information changes. Responses based on policies or documentation may remain valid for weeks or months, whereas information that changes more frequently requires a shorter lifetime to prevent outdated responses from being served.

For multi-tenant applications, the cache should also be partitioned appropriately to ensure data belonging to different users or organisations remains isolated.

Design principle 4: Observability and feedback

Semantic caching should not be treated as a one-time implementation. As applications evolve, the cache must be monitored continuously to ensure it continues delivering the expected benefits (Figure 4).

Developers should track metrics such as cache hit ratio, latency, token consumption and memory usage.

Design Principle 2 – Thresholds & Safety Checks

- Semantic cache is not binary; similarity is a continuum:
 - Too low threshold → false negatives (missed hits)
 - Too high threshold → false positives (wrong answers).
- Common pattern:
 - Distance threshold (e.g., cosine similarity > X)
 - Intent / classification filter (only cache specific intents)
 - Optional secondary classifier for “cacheable vs not cacheable” before LLM.

Figure 2: Threshold selection is essential for accurate semantic catching

Design Principle 3 – Cache Design & Eviction

- Eviction:
 - TTL tuned to business domain (hours–days for FAQs).
 - LRU/LFU when memory pressure is high
- Multi-tenant:
 - Partition by tenant or region to prevent cross-tenant leakage.

Figure 3: Effective cache design improves semantic caching performance

Design Principle 4 – Observability & Feedback

- Track:
 - Semantic hit ratio over time
 - False-positive & false-negative rates from human review
 - Token savings, latency per path (hit vs miss).
- Use traces to:
 - Re-tune thresholds and models
 - Identify new “cache-worthy” intents and non-cacheable edge cases.

Figure 4: Continuous monitoring improves semantic cache performance